

NoSQL I: Document Stores with MongoDB

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Part 1: Why NoSQL?
2. Part 2: The Document Model
3. Part 3: Schema Design – Embed or Reference?
4. Part 4: Querying MongoDB
5. Part 5: Hands-on Lab

Learning Objectives

By the end of this session, you will be able to:

- Explain **why** NoSQL databases appeared and name the main NoSQL families.
- Describe the **document model**: documents, collections, BSON, `_id`.
- Explain what **“schema-less”** really means (and what it does *not* mean).
- Decide when to **Embed** and when to **Reference** related data.
- Write basic MongoDB queries: `find`, updates and the **aggregation pipeline**.

What this session is NOT

One session cannot make you a MongoDB administrator. We focus on the **data model** and **design decisions**; operations (replica sets, sharding, backups) build on what you learned in Session 8.

Recap: Where are we?

Session 8: Distributed Databases

- Horizontal scaling: **Replication** (copies) and **Sharding** (splits).
- The cost of sharding: **cross-shard JOINs** are slow, foreign keys across shards are not supported.
- CAP: when the network breaks, choose **C** or **A**.

The Question for Today:

- If JOINs are the problem when data is spread over many servers…
- …what if we store data so that we **rarely need JOINs at all**?

This is the core idea of the Document Model.

Part 1: Why NoSQL?

1.1 Pain Points of the Relational Model

1. **Rigid schema:** adding a field to a 1-billion-row table needs a migration (`ALTER TABLE`). Startups change their data every week.
2. **Object-relational mismatch:** the application works with *objects* (a film with its actors and category), the database stores *flat rows* in 5 tables. Every screen needs JOINs to rebuild the object.
3. **Scaling out is hard:** JOINs and multi-table transactions do not shard well (Session 8).
4. **Varied data:** logs, JSON events from APIs, product attributes that differ for each product type.

AI Context

ML pipelines produce semi-structured data: model configs, experiment metadata, LLM chat histories, raw JSON from web APIs. Forcing them into fixed tables is painful.

1.2 NoSQL = “Not Only SQL”

NoSQL is a family of databases that do **not** use the relational table model. Each family is optimized for a specific access pattern.

Family	Data unit	Examples	Session
Document	JSON-like document	MongoDB, Couchbase	**9 (today)**
Key-Value	key → value	Redis, DynamoDB	10
Wide-Column	partition → rows	Cassandra, HBase	10
Graph	nodes + relationships	Neo4j	11
Vector	embedding vectors	Pinecone, Milvus, pgvector	12

Key Message

NoSQL does not *replace* SQL. Modern systems use several databases, each for what it does best: **Polyglot Persistence**.

1.3 ACID vs. BASE

ACID (Sessions 5–6)

- **A**tomicity, **C**onsistency, **I**solation, **D**urability.
- Correctness first.
- Typical of RDBMS: PostgreSQL.

BASE (many NoSQL systems)

- **B**asically **A**vailable, **S**oft state, **E**ventually consistent.
- Availability and scale first.
- Replicas *converge* over time.

Common Misconception

“NoSQL has no transactions” is outdated. MongoDB guarantees **atomic writes on a single document** and supports **multi-document ACID transactions** (since v4.0) – but they are slower, so good schema design tries to avoid needing them.

Part 2: The Document Model

2.1 Vocabulary: SQL vs. MongoDB

PostgreSQL	MongoDB
Database	Database
Table	**Collection**
Row	**Document**
Column	**Field**
Primary key	`_id` (always present, unique)
JOIN	**Embedding** or `$lookup`
Index (B-Tree)	Index (B-Tree)
`SELECT … WHERE`	`find({filter})`
`GROUP BY`	Aggregation pipeline (`$group`)

*Similar words – but a document can hold **nested objects** and **arrays**.*

2.2 Anatomy of a Document

```
{
  "_id": 1,
  "title": "Academy Dinosaur",
  "rating": "PG",
  "length": 86,
  "rental_rate": 0.99,
  "category": "Documentary",
  "special_features": ["Deleted Scenes",
                       "Behind the Scenes"],
  "actors": [
    { "actor_id": 1,  "name": "Penelope Guiness" },
    { "actor_id": 10, "name": "Christian Gable" }
  ]
}
```

- **Scalar fields:** string, number.
- **Array of values:** `special_features`.
- **Array of sub-documents:** `actors`.

BSON

MongoDB stores **Binary JSON**: faster to scan and has extra types (`Date`, `ObjectId`, `Decimal128`, int32/int64). Max size: **16 MB per document**.

2.3 One Film: 5 Tables vs. 1 Document

PostgreSQL (dvdrental)

```
SELECT f.title, c.name AS category,
       a.first_name || ' ' || a.last_name
FROM film f
JOIN film_category fc ON fc.film_id = f.film_id
JOIN category c       ON c.category_id = fc.
     category_id
JOIN film_actor fa    ON fa.film_id = f.film_id
JOIN actor a          ON a.actor_id = fa.actor_id
WHERE f.film_id = 1;
```

5 tables, 4 JOINs, 10 rows returned (one per actor) – the app must re-group them.

MongoDB

```
db.films.findOne({ _id: 1 })
```

1 collection, 0 JOINs, 1 document returned – already shaped like the object the app needs.

The Price

The actor's name is now **copied** into every film the actor plays in. If an actor changes name, we must update many documents.

2.4 Schema-less ≠ No Schema

Schema-on-write (PostgreSQL)

- The table defines the structure *before* data arrives.
- Bad data is rejected at `INSERT`.

Schema-on-read (MongoDB)

- Two documents in one collection may have **different fields**.
- The **application** interprets the structure when it reads.

```
// both are accepted in "films"
{ "_id": 7, "title": "Airplane Sierra" }
{ "_id": 8, "titel": "Airport Pollock",
  "imdb": { "score": 6.1 } }   // typo!
```

Optional validation brings rules back:

```
db.createCollection("films", { validator: {
  $jsonSchema: { required: ["title"],
    properties: { length: { bsonType: "number",
                            minimum: 1 } } } }
    })
```

Key Takeaway

There is always a schema – the question is **who enforces it**: the database or your code.

2.5 The `_id` Field and ObjectId

- Every document must have a unique `_id`. If you do not provide one, MongoDB generates an **ObjectId**, e.g. `ObjectId("6717a1f2c3d4e5f601234567")`.
- An ObjectId is **12 bytes**:

4 bytes: timestamp	5 bytes: random (per process)	3 bytes: counter

- Generated by the **client**, without asking the server ⇒ no central counter, works on many shards.
- Roughly **sorted by creation time**, unlike a random UUIDv4.

Link to Session 1

Remember `SERIAL` vs. `UUID`: a central sequence is a bottleneck in a distributed system. ObjectId solves the same problem as UUID.

2.6 PostgreSQL JSONB vs. MongoDB

Session 1: PostgreSQL also stores documents (`JSONB` + GIN index). So why MongoDB?

	PostgreSQL + JSONB	**MongoDB**
Main model	Tables; JSON is *one column*	Documents everywhere
Transactions	Full ACID, JOINs, FK	Single-doc atomic; multi-doc slower
Scale-out	Mostly vertical / extensions	Built-in sharding & replica sets
Query language	SQL + JSON operators	MQL + aggregation pipeline
Best when	Mostly relational data with a few flexible attributes	Mostly document-shaped data, fast-changing, very large

Rule of Thumb

If your data is 90% relational, keep PostgreSQL and use JSONB for the flexible 10%.

Part 3: Schema Design – Embed or Reference?

3.1 The Golden Rule

Design for Access Patterns

"Data that is **accessed together** should be **stored together**."

Relational design

1. Model the **entities**.
2. Normalize (3NF) to remove duplication.
3. Write any query later – JOINs handle it.

Document design

1. List the **queries/screens** first.
2. Shape documents so the frequent queries read **one document**.
3. Accept controlled duplication.

Two tools to connect related data: **Embedding** (put it inside) and **Referencing** (store the `_id`).

3.2 Embedding (Nesting)

```
// customers
{ "_id": 1, "first_name": "Mary",
  "last_name": "Smith",
  "address": {
     "street": "1913 Hanoi Way",
     "city": "Sasebo", "country": "Japan" } }
```

Pros

- One read returns everything.
- Writes to one document are **atomic** – no transaction needed.

Cons

- Document grows with the array; limit **16 MB**.
- Duplicated data must be updated in many places.
- Hard to query the child on its own (e.g. “all addresses in Japan” is fine, “all actors” needs `$unwind`).

Embed when

The child **belongs** to the parent, is read with it, and is **bounded** in size (1–1, 1–few).

3.3 Referencing (Linking)

```
// rentals: one document per rental
{ "_id": 1185,
  "customer_id": 1,    // -> customers._id
  "film_id": 611,      // -> films._id
  "rental_date": ISODate("2005-06-15T00:54:12Z"),
  "amount": 5.99 }
```

Pros

- No duplication, small documents.
- Child can be queried and updated on its own.
- Scales to millions of children.

Cons

- Needs a second query or `$lookup` to combine.
- **No foreign-key check**: MongoDB will accept `customer_id: 99999`.

Reference when

The child is **unbounded** (1-many, 1-millions), is shared by many parents (many-many), or is often accessed **on its own**.

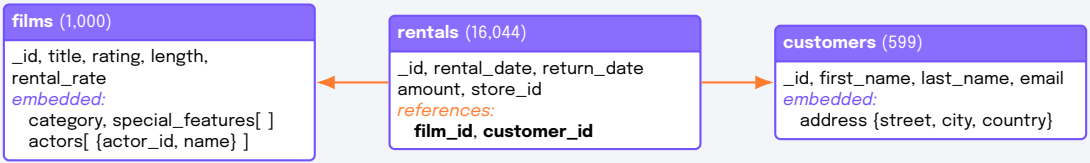
3.4 Decision Guide

Relationship	Example	Recommended
One-to-one	customer – address	**Embed** the object
One-to-few	film – special features, actors (5–10)	**Embed** an array
One-to-many	product – reviews (hundreds)	Reference, or embed only the **newest N** (Subset pattern)
One-to-squillions	customer – rentals, sensor – readings	**Reference from the child** (`customer_id` in each rental)
Many-to-many	students – courses	Array of `_ids` on one or both sides

Three questions to ask:

1. Is the child **always read with** the parent?
2. Can the number of children **grow without limit**?
3. Is the child **updated independently** or shared by other parents?

3.5 Case Study: dvdrental in MongoDB



- 15 PostgreSQL tables → **3 collections**.
- `actor`, `category`, `film_actor`, `film_category` are **embedded** in films (few, read together, rarely change).
- `address`, `city`, `country` are **embedded** in customers (1–1).
- `rental` + `payment` become **rentals**, which **reference** film and customer: a customer has an unbounded number of rentals.

3.6 Anti-Patterns to Avoid

1. Unbounded arrays

```
{ "_id": 1, "first_name": "Mary",
  "rentals": [ {...}, {...}, ... ] } // grows
     forever
```

- Each new rental rewrites a bigger document.
- Eventually hits the **16 MB** limit.

2. “Relational thinking” in MongoDB

- One collection per table, `$lookup` in every query.
- You get the costs of NoSQL (no FK, no JOIN optimizer) **and** the costs of JOINs.

3. Embedding hot, shared data

- A product price copied into 1M orders, but the price changes daily.

Rule

If you need `$lookup` on your **most frequent** query, revisit the design.

3.7 Think-Pair-Share (5 minutes)

Embed or Reference? Justify with the three questions.

1. A **user profile** with up to 3 shipping addresses.
2. An **e-commerce product** that may receive 50,000 reviews; the product page shows the 3 newest.
3. An **LLM chatbot**: each conversation has messages; a typical conversation has 20 messages, the app always loads a whole conversation.
4. An **IoT sensor** sending one reading every second for years.

3.8 Think–Pair–Share: Suggested Answers

1. **Embed** an array of addresses: one-to-few, always read with the profile.
2. **Reference** (separate `reviews` collection with `product_id`) + **embed the 3 newest** in the product: the **Subset pattern**.
3. **Embed** messages in the conversation: read together, bounded. (Very long chats: split into "buckets" of *N* messages.)
4. **Reference from the child**: one document per reading (or per hour – **Bucket pattern**); never an ever-growing array.

AI Context

Chat histories for LLM apps, experiment logs (MLflow-style runs with params and metrics) and feature documents are classic MongoDB workloads.

Part 4: Querying MongoDB

4.1 CRUD at a Glance

SQL	MongoDB (MQL)
`INSERT INTO films VALUES (...)`	`db.films.insertOne({ title: "X", ... })`
`SELECT * FROM films WHERE rating='PG'`	`db.films.find({ rating: "PG" })`
`SELECT title FROM films ...`	`db.films.find({...}, { title: 1, _id: 0 })`
`ORDER BY length DESC LIMIT 5`	`.sort({ length: -1 }).limit(5)`
`UPDATE films SET rental_rate=1.99 WHERE ...`	`db.films.updateOne({_id: 1}, { $set: { rental_rate: 1.99 } })`
`DELETE FROM films WHERE ...`	`db.films.deleteOne({ _id: 1 })`

Notation

Queries are themselves documents: `{ field: value }` means “field equals value”. Operators start with `$`.

4.2 `find()`: Filters, Nested Fields and Arrays

```
// comparison + AND (comma) + projection + sort
db.films.find(
  { rating: "PG-13", length: { $gt: 120 } },
  { _id: 0, title: 1, length: 1 }
).sort({ length: -1 })

// OR / IN
db.films.find({ rating: { $in: ["G", "PG"] } })

// dot notation into an embedded document
db.customers.find({ "address.country": "Japan" })

// match inside an array of sub-documents
db.films.find({ "actors.name": "Jayne Nolte" })

// array of values contains an element
db.films.find({ special_features: "Trailers" })
```

Operators

`$eq $ne $gt $gte $lt $lte`
`$in $nin $or $and $exists`

Dot notation

Field paths with a dot must be **quoted**: `"address.country"`.

Searching inside arrays needs no `$unwind` and no JOIN – this is the payoff of embedding.

4.3 Updates Are Atomic per Document

```
db.films.updateOne(
  { _id: 1 },                                  // which document
  { $set:  { rental_rate: 1.99 },              // change a field
    $push: { special_features: "Commentaries" }, // append to an array
    $inc:  { length: 2 } }                     // add to a number
)
```

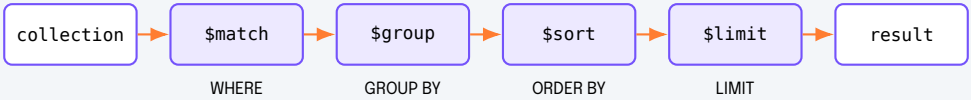
- All changes to **one document** are applied **all-or-nothing** (Atomicity).
- With good embedding, most business operations touch one document ⇒ no transaction needed.
- `updateMany` changes every matching document; `$pull` removes from an array.

Careful

`updateOne({_id:1}, { rental_rate: 1.99 })` without `$set` is an error in modern drivers; the old `update()` **replaced the whole document**!

4.4 The Aggregation Pipeline

Documents flow through **stages**, like a Unix pipe: each stage transforms the stream and passes it on.



Stage	Role / SQL equivalent
`$match`	filter documents (`WHERE`, or `HAVING` after `$group`)
`$group`	group + aggregate: `$sum`, `$avg`, `$max` (`GROUP BY`)
`$project`	choose / compute fields (`SELECT` list)
`$unwind`	one output document per array element
`$lookup`	combine with another collection (`LEFT JOIN`)

Remember Session 3: put `$match` **early** so later stages process fewer documents (filter push-down).

4.5 Pipeline Example: Top Spenders



MongoDB

```
db.rentals.aggregate([
  { $group: { _id: "$customer_id",
              total: { $sum: "$amount" },
              rentals: { $sum: 1 } } },
  { $match: { rentals: { $gte: 5 } } },
  { $sort:  { total: -1 } },
  { $limit: 3 }
])
```

PostgreSQL

```
SELECT customer_id,
       SUM(amount) AS total,
       COUNT(*)    AS rentals
FROM rentals
GROUP BY customer_id
HAVING COUNT(*) >= 5
ORDER BY total DESC
LIMIT 3;
```

- `"$amount"` (with `$`) means “the value of field `amount`”; `"amount"` is just a string.
- `_id` in `$group` is the **grouping key**.
- `$match` after `$group` plays the role of `HAVING`.

4.6 $lookup: the “JOIN” of MongoDB

```
db.rentals.aggregate([
  { $match: { customer_id: 1 } },
  { $lookup: { from: "films",          // the other collection
               localField: "film_id",  // field in rentals
               foreignField: "_id",    // field in films
               as: "film" } },         // output: an ARRAY of matches
  { $project: { _id: 0, rental_date: 1, title: { $first: "$film.title" } } }
])
```

- Behaves like a **LEFT OUTER JOIN**: no match ⇒ empty array `[]`.
- The result is an **array**: flatten with `$unwind` or `$first`.
- Index the `foreignField`, or every input document scans the other collection (Session 3: Nested Loop without index).

Design Smell

`$lookup` is fine for reports. If your **hot path** needs it, consider embedding.

4.7 Indexes and `explain()`

```
db.rentals.find({ customer_id: 3 }).explain()
// winningPlan: { stage: "COLLSCAN" }

db.rentals.createIndex({ customer_id: 1 })

db.rentals.find({ customer_id: 3 }).explain()
// winningPlan: { stage: "FETCH",
//   inputStage: { stage: "IXSCAN",
//     indexName: "customer_id_1" } }
```

MongoDB	PostgreSQL
`COLLSCAN`	Seq Scan
`IXSCAN` + `FETCH`	Index Scan
`_id` index	Primary key
Compound index	Composite index
Multikey index	GIN on array

Same Principles as Session 2

Indexes are B-Trees: they speed up reads, cost extra writes and space. An index on an **array field** (e.g. `actors.name`) is a **multikey index**: one entry per element.

4.8 When to Choose MongoDB

Good fit

- Product catalogs with varied attributes.
- Content management, user profiles.
- Event / log data, IoT (with bucketing).
- ML metadata, LLM chat history.

Poor fit

- Banking ledgers (multi-entity transactions).
- Heavy ad-hoc reporting with many JOINs.
- Highly connected data → **Graph DB**.
- Strict integrity rules (FK, CHECK).

Remember Session 8 (CAP)

A MongoDB replica set has one primary for writes; during a partition the minority side cannot accept writes ⇒ by default it behaves as a **CP** system.

Part 5: Hands-on Lab

5.1 Lab Setup: Mongo Playground (no installation)



1. Open **`https://mongoplayground.net`** in your browser. No account needed.
2. In the template drop-down, choose **“bson multiple collections”**.
3. **Configuration** (left pane): delete the sample and paste the content of `playground_config.js` (from the course page). It starts with `db={ "films": [...], "customers": [...], "rentals": [...] }`.
4. **Query** (middle pane): write one query, e.g. `db.films.find({_id: 1})`.
5. Click **Run**. The result appears on the right.
6. Click **Share** to get a link ⇒ this is how you **submit** each exercise.

Playground limits

Only `find(filter, projection)`, `aggregate([...])` and `update(...)` are supported – **no** `.sort()`/`.limit()` after `find`: use `aggregate` instead. Max 100 documents per collection; updates are not saved between runs.

5.2 The Lab Dataset (a slice of dvdrental)

Collection	Docs	Shape
`films`	52	`_id, title, description, rating, length, rental_rate, category, special_features[ ], actors[{actor_id, name}]`
`customers`	8	`_id, first_name, last_name, email, store_id, active, address{street, district, city, country, phone}`
`rentals`	48	`_id, customer_id, film_id, store_id, rental_date, return_date, amount`

- Customers 1–8, their first 6 paid rentals each, and every film they rented (+ 5 films nobody rented).
- The full dataset (1,000 films, 599 customers, 16,044 rentals) is also provided as `full_*.json` for MongoDB Atlas.

5.3 Guided Lab – Part A: `find()` (15 min)



L1. List the titles and lengths of **PG-13** films longer than **120** minutes. Then show them **longest first** (Playground: rewrite with `aggregate`: `$match`, `$project`, `$sort`).

L2. Which customers live in the **United States**? Show first name, last name and city.

L3. Which films does actor **Jayne Nolte** appear in? Show titles only.

Hints

L1: `$gt`, projection, `$sort`. L2: dot notation. L3: query inside an array of sub-documents.

5.4 Part A: Solutions

```
// L1 (find)
db.films.find({ rating: "PG-13", length: { $gt: 120 } },
  { _id: 0, title: 1, length: 1 })
// L1 sorted (mongosh: add .sort({length: -1}))
db.films.aggregate([
  { $match: { rating: "PG-13", length: { $gt: 120 } } },
  { $project: { _id: 0, title: 1, length: 1 } },
  { $sort: { length: -1 } } ])

// L2
db.customers.find({ "address.country": "United States" },
  { _id: 0, first_name: 1, last_name: 1,
    "address.city": 1 })

// L3
db.films.find({ "actors.name": "Jayne Nolte" }, { _id: 0,
     title: 1 })
```

L1 (5 films): Jacket Frisco 181, Idaho Love 172, Telemark Heartbreakers 152, Detective Vision 143, Streak Ridgemont 132.

L2: Patricia Johnson (San Bernardino), Jennifer Davis (Laredo).

L3 (6 films): Anaconda Confessions, Bedazzled Married, Hysterical Grail, Ice Crossing, Strangers Graffiti, Telemark Heartbreakers.

5.5 Guided Lab – Part B: Aggregation (25 min)

L4. Count films per **category**; show the top 3 (break ties by category name).

L5. Total amount paid by each customer (round to 2 decimals); show the top 3.

L6. Extend L5: show the customer's **full name** instead of the id (`$lookup`).

L7. Film 1 now costs **1.99** and gets the feature **"Commentaries"** – one update statement.

Hints

L4: `$group` with `_id: "$category"`. L5: `$sum: "$amount"`, `$round`. L6: `$lookup` + `$unwind` + `$concat`. L7: `$set` + `$push`.

5.6 Part B: Solutions (L4–L5)

```
// L4
db.films.aggregate([
  { $group: { _id: "$category", n: { $sum: 1 } } },
  { $sort:  { n: -1, _id: 1 } },
  { $limit: 3 }
])

// L5
db.rentals.aggregate([
  { $group: { _id: "$customer_id",
              total: { $sum: "$amount" } } },
  { $project: { total: { $round: ["$total", 2] } } },
  { $sort: { total: -1 } },
  { $limit: 3 }
])
```

L4: Drama 6, Animation 5, Classics 5. (Horror and Sports also have 5 – the tie-break on `_id` decides.)

L5: customer 3 → 35.94, customer 2 → 29.94, customer 1 → 27.94.

Why `$round`?

Without it, customer 6 shows `12.940000000000001`: `amount` is a binary double. For money, use `Decimal128` (like `NUMERIC` in PostgreSQL).

5.7 Part B: Solutions (L6–L7)

```
// L6
db.rentals.aggregate([
  { $group: { _id: "$customer_id",
              total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }, { $limit: 3 },
  { $lookup: { from: "customers", localField: "_id",
               foreignField: "_id", as: "cust" } },
  { $unwind: "$cust" },
  { $project: { _id: 0, total: { $round: ["$total", 2] },
      name: { $concat: ["$cust.first_name", " ",
                        "$cust.last_name"] } } } ])
// L7  (Playground: update; mongosh: updateOne)
db.films.update({ _id: 1 },
  { $set:  { rental_rate: 1.99 },
    $push: { special_features: "Commentaries" } })
```

L6: Linda Williams 35.94, Patricia Johnson 29.94, Mary Smith 27.94.

Why `$lookup` after `$limit`? Only 3 documents need to be joined instead of 48 – filter and reduce first, join last.

L7: `special_features` becomes `["Deleted Scenes", "Behind the Scenes", "Commentaries"]`.

5.8 Self-Practice (20 min)

Submit one **Playground Share link** per exercise (E1–E3); E4–E5 on paper or in a doc.

- **E1.** Which film was rented **most often**? Show its title and the number of rentals.
- **E2.** List the films that were **never rented** (hint: `$lookup` from films to rentals, then `$size`).
- **E3.** Two actors in dvdrental are both called **"Susan Davis"**. Count films per actor using `actors.name`, then using `actors.actor_id`. Explain the difference.
- **E4.** **Design:** the store adds customer **reviews** of films (a popular film may get 10,000 reviews; the film page shows the 3 newest). Where do you store them? Draw the documents and justify with the three questions of 3.4.
- **E5.** Write the PostgreSQL query (on the original dvdrental) that returns the same result as L6.

Summary

- NoSQL = **Not Only SQL**: each family fits a specific access pattern; real systems are **polyglot**.
- A **document** (BSON) holds nested objects and arrays; documents live in **collections**.
- **Schema-less** means schema-on-read: the application (or a validator) enforces structure.
- **Design for access patterns**: embed what is read together and bounded; reference what is unbounded, shared or used alone.
- MQL: `find` with filters/dot notation; the **aggregation pipeline** (`$match`, `$group`, `$lookup`, …) maps to SQL clauses.
- Same physical principles as PostgreSQL: B-Tree indexes, `COLLSCAN` vs. `IXSCAN`, filter early.

Next session: What if we only need "give me the value for this key" – in microseconds? → ***Redis*** *and* ***Cassandra****.*